Quá trình hình thành đề tài: Hệ thống phát hiện cháy bằng thị giác máy tính

---

1. Vấn đề thực tế

Mỗi năm, hỏa hoạn gây ra hàng nghìn ca tử vong và thiệt hại tài sản khổng lồ — phần lớn không phải vì lửa bùng lên quá mạnh, mà vì **phát hiện quá muộn**.

> **Ví dụ:** Một kho hàng bốc khói lúc 2 giờ sáng. Không ai ở đó. Camera an ninh đang ghi hình nhưng không ai xem. Đến khi báo động khói kích hoạt, lửa đã lan rộng 15 phút.

Câu hỏi đặt ra: *Tại sao hệ thống camera sẵn có lại không thể tự động nhận diện và cảnh báo?*

---

2. Vì sao vấn đề vẫn tồn tại?

2.1 Cơ chế phát hiện hiện tại còn thô sơ

Các hệ thống phổ biến hiện nay dựa trên:

Cơ chế	Nguyên lý	Hạn chế
Đầu báo nhiệt	Cảm biến nhiệt độ	Chỉ phát hiện khi lửa đã gần, không nhìn xa
Đầu báo khói ion	Hạt khói phá vỡ dòng điện	Nhạy với bụi, hơi nước — báo nhầm nhiều
Camera an ninh thông thường	Ghi hình thụ động	Cần con người theo dõi liên tục

2.2 Hạn chế cốt lõi

Không có trí tuệ phân tích hình ảnh theo thời gian thực. Camera có thể "thấy" nhưng không "hiểu". Khi lửa xuất hiện trong khung hình, không có thành phần nào tự động nhận ra và phản ứng.

2.3 Nếu dùng AI ngây thơ — vẫn chưa đủ

Giả sử ta huấn luyện một mô hình nhận diện lửa rồi chạy từng frame — **vẫn còn một vấn đề nghiêm trọng**: mô hình phát hiện lửa trên từng khung hình riêng lẻ sẽ bị nhiễu bởi:

- Ánh đèn phản chiếu qua kính
- Artifact nén video (vùng màu cam/đỏ)
- Ánh nắng chiếu thẳng vào ống kính trong 1–2 frame

> **Ví dụ minh họa:** Model dự đoán `fire, conf=0.72` tại frame 103, nhưng frame 102 và 104 hoàn toàn bình thường — đây là false positive điển hình từ ánh đèn xe chiếu vào camera đêm.

Kết quả: hệ thống kích báo động giả liên tục → người vận hành mất niềm tin → tắt báo động → mục đích ban đầu bị triệt tiêu.

---

3. Giải pháp đề xuất

Vì sao cần phát hiện hình ảnh bằng deep learning?

> **Vì** camera có độ phủ rộng, hoạt động 24/7, và thị giác máy tính hiện đại đã đủ mạnh để nhận diện lửa/khói trong điều kiện thực tế — nhưng cần đúng kiến trúc.

→ **Giải pháp:** Dùng **YOLOv8** — mô hình object detection thời gian thực — để phân tích từng frame từ camera, nhận diện hai lớp: `fire` và `smoke`.

```
Frame từ camera
       ↓
  YOLOv8Detector.predict()
       ↓
  [Detection(class="fire", conf=0.81, box=(120,45,310,280)), ...]
```

Vì sao cần lọc thời gian (temporal filtering)?

> **Vì** một phát hiện đơn lẻ không đủ tin cậy — lửa thật xuất hiện **liên tục trong nhiều frame liên tiếp**, còn false positive thường chỉ xuất hiện **1–2 frame rồi biến mất**.

→ **Giải pháp:** Dùng `TemporalFilter` với cửa sổ trượt (sliding window): chỉ xác nhận cháy khi có **ít nhất** `min_det` **frame phát hiện trong vòng** `window_seconds` **giây**.

```
Cấu hình mặc định:
  conf_threshold  = 0.45   # chỉ đếm frame nếu confidence ≥ 0.45
  window_seconds  = 1.0    # nhìn lại trong 1 giây gần nhất
  min_detections  = 3      # cần ít nhất 3 frame để xác nhận
  cooldown_seconds = 30.0  # chờ 30 giây trước khi phát lại cảnh báo
```

Ví dụ so sánh:

Tình huống	Frame phát hiện trong 1s	Kết quả
Ánh đèn xe chiếu vào	1 frame (`conf=0.72`)	`SUSPECTED` → quay về `IDLE` , **không báo**
Lửa thật bùng lên	6 frame liên tiếp (`conf ≥ 0.6`)	`SUSPECTED` → `CONFIRMED` , **phát cảnh báo**

Vì sao cần state machine 4 trạng thái?

> **Vì** thực tế không nhị phân. Lửa có thể bị che khuất tạm thời (khói dày, vật cản), mô hình có thể miss vài frame — không nên tắt cảnh báo ngay khi mất detection.

→ **Giải pháp:** `AlertState` quản lý 4 trạng thái rõ ràng:

```
IDLE ──(có detection)──→ SUSPECTED
SUSPECTED ──(đủ evidence)──→ CONFIRMED  [phát alert]
CONFIRMED ──(mất detection)──→ COOLDOWN
COOLDOWN ──(hết timer + detection quay lại)──→ CONFIRMED [phát lại]
COOLDOWN ──(hết timer + không còn detection)──→ IDLE
```

Vì sao cần giao diện web và cảnh báo email?

> **Vì** hệ thống phải dùng được bởi người không chuyên kỹ thuật, và cảnh báo phải đến tay người chịu trách nhiệm ngay cả khi họ không ở trước màn hình.

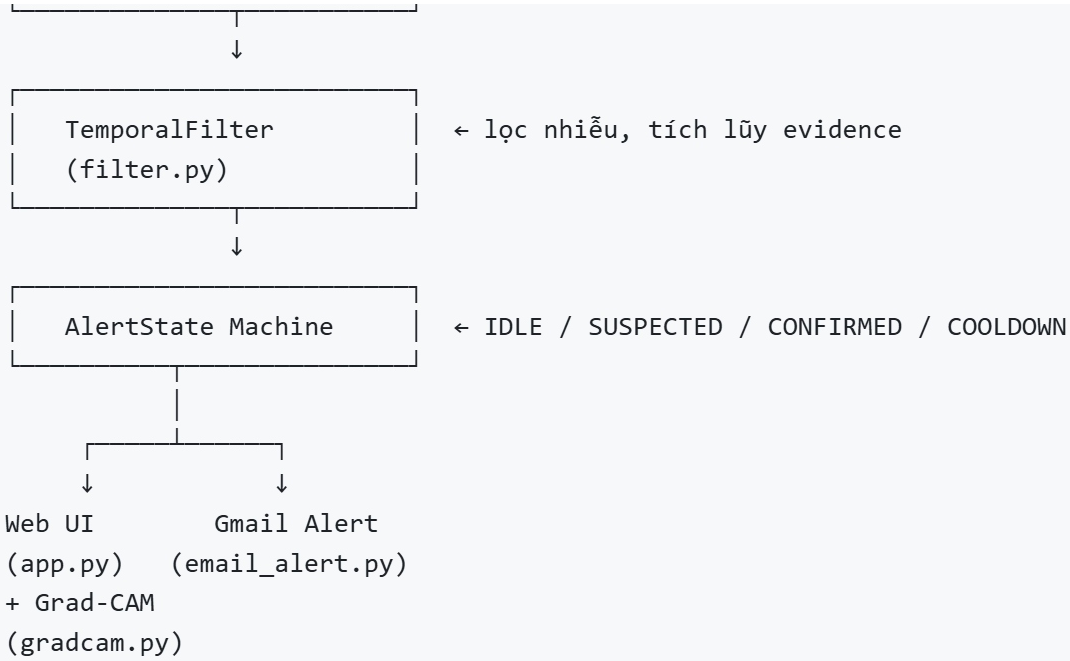
→ **Giải pháp:**

- **FastAPI web app** cho phép upload ảnh, dán URL, hoặc thử ảnh mẫu — trả về kết quả ngay trên trình duyệt.
- **Grad-CAM** trực quan hóa vùng mà mô hình "nhìn vào" để đưa ra quyết định — tăng khả năng giải thích.
- **Gmail alert** gửi email kèm ảnh phát hiện đến địa chỉ được cài đặt khi có cháy xác nhận.

---

4. Tổng kết kiến trúc giải pháp

```
┌──────────────────────────────────────────────────────┐
│                    Nguồn đầu vào                     │
│   Camera thời gian thực | Ảnh upload | URL ảnh       │
└──────────────┬───────────────────────────────────────┘
               ↓
┌──────────────────────────┐
│   YOLOv8Detector         │  ← phát hiện fire/smoke theo frame
│   (model.py)             │
└──────────────────────────┘
```



Kết quả kỳ vọng: Hệ thống có thể chạy trên máy tính thông thường (CPU) hoặc GPU, phát hiện cháy trong vài trăm millisecond mỗi frame, và chỉ phát cảnh báo khi có đủ bằng chứng tích lũy theo thời gian — giảm đáng kể false positive so với phương pháp per-frame đơn thuần.